Polimorfisme & Abstrak

Polimorfisme

Polimorfisme berarti satu antarmuka dengan banyak implementasi. Istilah ini berasal dari bahasa Yunani dan dari dua suku kata yaitu *poly* berarti banyak dan *morphe* berarti bentuk. Dalam OOP, kode yang sama dapat bekerja dengan objek bertipe berbeda, selama objek-objek itu memiliki method yang diharapkan.

Mengapa polimorfisme penting?

- **Fleksibilitas Kode yang Tinggi:** Memungkinkan fungsi atau metode menerima berbagai jenis objek selama objek tersebut memiliki nama metode yang sesuai (dikenal sebagai *duck typing* di python).
- ***Code Reusability*:** Kode dapat digunakan secara berulang sehingga memudahkan programmer dalam melakukan perubahan terhadap kode tanpa mengubah keseluruhan program
- ***Maintainability:*** Saat ingin menambah fitur atau kelas baru, cukup membuat kelas turunan dengan metode yang menyesuaikan.

Jenis-Jenis Polimorfisme

Polimorfisme Via Pewarisan

Pada jenis ini, kelas anak meng-override method dari kelas induk, sehingga pemanggilan method yang sama menghasilkan perilaku berbeda tergantung tipe objeknya saat runtime (disebut *dynamic dispatch*). Pada perbankan, polimorfisme ini terjadi karena adanya pewarisan kelas Tabungan dan kelas Deposito dari kelas AkunBank. Masing-masing method kelas turunan memiliki method yang sama, namun output yang berbeda.

```
class AkunBank:
    def __init__(self, nama, saldo):
        self.nama = nama
```

```
        self.saldo = saldo

    def hitung_bunga(self):
        return 0

class Tabungan(AkunBank):
    def hitung_bunga(self):
        # Bunga 2% untuk tabungan biasa
        return self.saldo * 0.02

class Deposito(AkunBank):
    def hitung_bunga(self):
        # Bunga 6% untuk deposito
        return self.saldo * 0.06

# Polimorfisme: Satu loop, perilaku berbeda
daftar_akun = [Tabungan("Budi", 1000000), Deposito("Sari", 5000000)]

for akun in daftar_akun:
    print(f"Bunga {akun.nama}: Rp {akun.hitung_bunga():,.2f}")

# Output:
# Bunga Budi: Rp 20,000.00
# Bunga Sari: Rp 300,000.00
```

Duck Typing (Polimorfisme Tanpa Pewarisan)

Duck typing berfokus pada method apa yang dimiliki sebuah objek, bukan tipe objeknya atau hubungan pewarisannya. Selama objek memiliki method yang dibutuhkan, ia bisa digunakan secara bergantian. Pada perbankan, polimorfisme ini terjadi karena RekeningTabungan dan RekeningGiro sama-sama memiliki method tarik_uang(). Fungsi proses_penarikan() tidak perlu tahu jenis rekeningnya, bahkan kedua kelas itu tidak memiliki hubungan pewarisan sama sekali.

```
class RekeningTabungan:
```

```
    def __init__(self, saldo):
        self.saldo = saldo

    def tarik_uang(self, jumlah):
        if jumlah <= self.saldo:
            self.saldo -= jumlah
            print(f"Penarikan Rp{jumlah:,} berhasil.")
            print(f"Saldo tersisa: Rp{self.saldo:,}")
        else:
            print("Saldo tidak cukup.")


class RekeningGiro:
    def __init__(self, saldo):
        self.saldo = saldo

    def tarik_uang(self, jumlah):
        if jumlah <= self.saldo:
            self.saldo -= jumlah
            print(f"Penarikan Rp{jumlah:,} dari rekening giro berhasil.")
            print(f"Saldo tersisa: Rp{self.saldo:,}")
        else:
            print("Saldo tidak cukup.")

# duck typing
def proses_penarikan(rekening, jumlah):
    rekening.tarik_uang(jumlah)


tabungan = RekeningTabungan(5_000_000)
giro = RekeningGiro(10_000_000)

proses_penarikan(tabungan, 1_000_000)
proses_penarikan(giro, 2_000_000)

# Output:
# Penarikan Rp1,000,000 berhasil.
# Saldo tersisa: Rp4,000,000
# Penarikan Rp2,000,000 dari rekening giro berhasil.
# Saldo tersisa: Rp8,000,000
```

Abstract Base Class (ABC)

ABC memaksa kelas anak mengimplementasikan method tertentu sebagai kontrak yang pasti. Ini mencegah program meledak saat *runtime*, karena error akan langsung terdeteksi saat objek baru dibuat jika implementasinya tidak lengkap.

```python
from abc import ABC, abstractmethod

class Transaksi(ABC):
    @abstractmethod
    def validasi(self):
        """Wajib diimplementasikan oleh kelas anak."""
        pass

    @abstractmethod
    def eksekusi(self):
        """Wajib diimplementasikan oleh kelas anak."""
        pass

class TarikTunai(Transaksi):
    def __init__(self, saldo_akun, nominal):
        self.saldo = saldo_akun
        self.nominal = nominal

    def validasi(self):
        if self.nominal > self.saldo:
            return False
        return True

    def eksekusi(self):
        if self.validasi():
            self.saldo -= self.nominal
            print(f"Tarik tunai berhasil. Sisa saldo: {self.saldo}")
        else:
            print("Saldo tidak cukup!")

# Jika kita membuat kelas tapi lupa implementasi method dari ABC:
class TransferGagal(Transaksi):
    def validasi(self):
        return True
    # Lupa implementasi eksekusi()!
```

```
transaksi_sukses = TarikTunai(100000, 50000)
transaksi_sukses.eksekusi()
# transaksi_gagal = TransferGagal()

try:
    transaksi_gagal = TransferGagal()   # Error! eksekusi() belum
diimplementasikan
except TypeError as e:
    print(f"Error: {e}")
    # Can't instantiate abstract class TransferGagal without an implementation
    # for abstract method 'eksekusi'
```

Perhatikan bahwa error muncul saat objek dibuat, bukan saat eksekusi() dipanggil. Dengan begitu, kelas yang tidak lengkap langsung ketahuan sebelum program berjalan lebih jauh dan sebelum uang nasabah diproses.

Operator Overloading

Python memungkinkan kita mendefinisikan ulang perilaku operator (seperti +, -, >) untuk kelas buatan sendiri, yang juga merupakan bentuk polimorfisme.

```
class Rekening:
    def __init__(self, no_rek, saldo):
        self.no_rek = no_rek
        self.saldo = saldo

    # Mendefinisikan operator + (__add__)
    def __add__(self, rekening_lain):
        total_saldo = self.saldo + rekening_lain.saldo
        return f"Total gabungan saldo: Rp {total_saldo:,}"

    # Mendefinisikan operator > (__gt__)
    def __gt__(self, rekening_lain):
        return self.saldo > rekening_lain.saldo

   # Tambahkan di dalam kelas Rekening:
```

```
    # Mendefinisikan operator - (__sub__)
    def __sub__(self, rekening_lain):
        return self.saldo - rekening_lain.saldo

    # Mendefinisikan operator == (__eq__)
    def __eq__(self, rekening_lain):
        return self.no_rek == rekening_lain.no_rek

    # Mendefinisikan tampilan saat di-print (__str__)
    def __str__(self):
        return f"Rekening {self.no_rek} | Saldo: Rp {self.saldo:,}"

rek_utama = Rekening("123", 5000000)
rek_cadangan = Rekening("456", 2000000)

print(rek_utama + rek_cadangan) # Memanggil __add__
print(rek_utama > rek_cadangan) # Memanggil __gt__, Output: True

print(rek_utama - rek_cadangan)   # 3000000      <- __sub__ (selisih saldo)
print(rek_utama == rek_cadangan)  # False        <- __eq__ (no. rekening
beda)
print(rek_utama)                  # Rekening 123 | Saldo: Rp 5,000,000  <-
__str__

# Output:
# Total gabungan saldo: Rp 7,000,000
# True
```

Tabel Operator dan Magic Method-nya:

Operator	*Magic Method*	Contoh
+	__add__	rek_a + rek_b
-	__sub__	rek_a - rek_b
==	__eq__	rek_a == rek_b

<	__lt__	rek_a < rek_b
>	__gt__	rek_a > rek_b
len()	__len__	len(bank)
str()	__str__	str(rek_a)

Simulasi Method Overloading di Python

Python tidak mendukung method overloading (seperti Java/C++) secara native. Sebagai gantinya, Python menggunakan default argument dan ***args**, sehingga satu method bisa dipanggil dengan beberapa "versi" parameter.

```
class KalkulatorBunga:
    def hitung(self, saldo, persen=2, bulan=1):
        # Satu method, beberapa "versi" pemanggilan
        return saldo * (persen / 100) * (bulan / 12)

    def total_setoran(self, *jumlah):
        # Menerima setoran sebanyak apapun
        return sum(jumlah)


kalkulator = KalkulatorBunga()

print(f"{kalkulator.hitung(1_000_000):,.2f}")         # 1,666.67  (bunga
2%, 1 bulan)
print(f"{kalkulator.hitung(1_000_000, 6):,.2f}")      # 5,000.00  (bunga
6%, 1 bulan)
print(f"{kalkulator.hitung(1_000_000, 6, 12):,.2f}")  # 60,000.00 (bunga
6%, 12 bulan)

print(kalkulator.total_setoran(500_000))                    # 500000
print(kalkulator.total_setoran(500_000, 250_000, 100_000))  # 850000
```

isinstance() dalam Konteks Polimorfisme

Saat menggunakan polimorfisme, kadang kita perlu tahu tipe suatu objek. Gunakan isinstance(), jangan gunakan type() karena type() tidak mengenali pewarisan.

```python
class AkunBank:
    def __init__(self, nama, saldo):
        self.nama = nama
        self.saldo = saldo

class Tabungan(AkunBank):
    pass

class Deposito(AkunBank):
    pass


budi = Tabungan("Budi", 1_000_000)

# Cara BENAR: isinstance() mengenali hierarki
print(isinstance(budi, Tabungan))   # True
print(isinstance(budi, AkunBank))   # True   <- Tabungan mewarisi AkunBank
print(isinstance(budi, Deposito))   # False

# Cara KURANG TEPAT: type() tidak mengenali hierarki
print(type(budi) == Tabungan)       # True
print(type(budi) == AkunBank)       # False  <- padahal Budi punya akun
bank!
```

Pola Umum: Dispatch Berdasarkan Tipe

```python
def proses_akun(akun):
    if isinstance(akun, Deposito):
        print(f"  {akun.nama}: deposito dikunci sampai jatuh tempo.")
    elif isinstance(akun, Tabungan):
        print(f"  {akun.nama}: tabungan bisa ditarik kapan saja.")
    elif isinstance(akun, AkunBank):
        print(f"  {akun.nama}: akun umum.")
```

Jebakan 1: Urutan isinstance() penting. Cek kelas anak lebih dulu, baru kelas induk. Jika AkunBank dicek paling awal, semua objek akan masuk ke cabang itu.

Jebakan 2: Jika terlalu banyak isinstance() di satu tempat, itu sinyal desain kelas perlu diperbaiki. Pindahkan logika khusus ke dalam method kelas masing-masing, bukan di luar.

Polimorfisme dengan Fungsi Built-in Python

Polimorfisme bukan hanya untuk kode kita sendiri, fungsi built-in Python pun polimorfis.

```
# len() bekerja untuk berbagai tipe
print(len("Bank Nusantara"))        # 14  <- string
print(len([1, 2, 3, 4, 5]))         # 5   <- list
print(len({"a": 1, "b": 2}))        # 2   <- dict
```

Kita juga bisa membuat kelas sendiri yang kompatibel dengan len():

```
class Bank:
    def __init__(self, nama):
        self.nama = nama
        self._nasabah = []

    def tambah_nasabah(self, nama):
        self._nasabah.append(nama)

    def __len__(self):
        return len(self._nasabah)

    def __str__(self):
        return f"{self.nama} ({len(self)} nasabah)"


bank = Bank("Bank Nusantara")
for n in ["Budi", "Sari", "Rudi", "Ayu"]:
    bank.tambah_nasabah(n)

print(len(bank))   # 4
print(bank)        # Bank Nusantara (4 nasabah)
```

sorted(), ***max()***, dan ***min()*** akan bekerja otomatis jika kelas mengimplementasikan __lt__:

```
class RekeningNasabah:
    def __init__(self, nama, saldo):
        self.nama = nama
        self.saldo = saldo
```

```
    def __lt__(self, other):   # kurang dari
        return self.saldo < other.saldo

    def __str__(self):
        return f"{self.nama} (Saldo: Rp {self.saldo:,})"


daftar = [
    RekeningNasabah("Budi", 3_500_000),
    RekeningNasabah("Sari", 8_500_000),
    RekeningNasabah("Rudi", 1_200_000),
]

# sorted() otomatis menggunakan __lt__
for r in sorted(daftar, reverse=True):
    print(f"  {r}")
# Output (dari saldo terbesar):
#   Sari (Saldo: Rp 8,500,000)
#   Budi (Saldo: Rp 3,500,000)
#   Rudi (Saldo: Rp 1,200,000)

print(max(daftar))   # Sari (Saldo: Rp 8,500,000)
print(min(daftar))   # Rudi (Saldo: Rp 1,200,000)
```

Studi Kasus: Sistem Transaksi Bank

Semua konsep berpadu dalam satu sistem yang realistis: sebuah mesin ATM yang bisa memproses berbagai jenis transaksi (setor, tarik, transfer) menggunakan polimorfisme. Kita bangun bertahap.

Langkah 1: Rekening dan Kontrak Transaksi (ABC)

```
from abc import ABC, abstractmethod


class RekeningBank:
    def __init__(self, no_rek, pemilik, saldo=0):
        self.no_rek = no_rek
```

```
        self.pemilik = pemilik
        self.saldo = saldo

    def __str__(self):
        return f"{self.no_rek} a.n. {self.pemilik} | Saldo:
Rp{self.saldo:,}"


class TransaksiBank(ABC):
    """Kelas abstrak: kontrak untuk semua jenis transaksi."""
    jenis = "Transaksi"

    def __init__(self, rekening, nominal):
        self.rekening = rekening
        self.nominal = nominal

    @abstractmethod
    def validasi(self):
        """Kembalikan True jika transaksi boleh dilakukan."""
        pass

    @abstractmethod
    def eksekusi(self):
        """Ubah saldo sesuai jenis transaksi."""
        pass

    def jalankan(self):
        """Method konkret: alur yang sama untuk semua transaksi."""
        if self.validasi():
            self.eksekusi()
            status = "BERHASIL"
        else:
            status = "GAGAL"
        print(f"  {self.jenis:<12} Rp{self.nominal:>10,} -> {status}")
```

Method **jalankan()** tidak perlu tahu jenis transaksinya. Ia hanya memanggil **validasi()** dan **eksekusi()**, dan setiap kelas anak menjawab dengan caranya sendiri. Inilah polimorfisme.

Langkah 2: Jenis-Jenis Transaksi

```python
class Setor(TransaksiBank):
    jenis = "Setor Tunai"

    def validasi(self):
        return self.nominal > 0

    def eksekusi(self):
        self.rekening.saldo += self.nominal


class Tarik(TransaksiBank):
    jenis = "Tarik Tunai"

    def validasi(self):
        return 0 < self.nominal <= self.rekening.saldo

    def eksekusi(self):
        self.rekening.saldo -= self.nominal


class Transfer(TransaksiBank):
    jenis = "Transfer"
    BIAYA_ADMIN = 2500

    def __init__(self, rekening, nominal, tujuan):
        super().__init__(rekening, nominal)
        self.tujuan = tujuan

    def validasi(self):
        total = self.nominal + self.BIAYA_ADMIN
        return self.nominal > 0 and total <= self.rekening.saldo

    def eksekusi(self):
        self.rekening.saldo -= (self.nominal + self.BIAYA_ADMIN)
        self.tujuan.saldo += self.nominal
```

Langkah 3: Mesin ATM dan Demonstrasi

```
class MesinATM:
    def __init__(self, lokasi):
        self.lokasi = lokasi

    def proses(self, daftar_transaksi):
        print(f"\nATM {self.lokasi}")
        for transaksi in daftar_transaksi:
            transaksi.jalankan()   # tidak peduli jenis transaksinya


budi = RekeningBank("123", "Budi", 1_000_000)
sari = RekeningBank("456", "Sari", 500_000)

atm = MesinATM("Kampus Gunung Kelua")
atm.proses([
    Setor(budi, 500_000),
    Tarik(budi, 200_000),
    Transfer(budi, 300_000, sari),
    Tarik(budi, 5_000_000),   # melebihi saldo
])

print()
print(budi)
print(sari)

# Output:
# ATM Kampus Gunung Kelua
#   Setor Tunai  Rp   500,000 -> BERHASIL
#   Tarik Tunai  Rp   200,000 -> BERHASIL
#   Transfer     Rp   300,000 -> BERHASIL
#   Tarik Tunai  Rp 5,000,000 -> GAGAL
#
# 123 a.n. Budi | Saldo: Rp997,500
# 456 a.n. Sari | Saldo: Rp800,000
```

Langkah 4: Menambah Fitur Tanpa Mengubah Kode Lama

Bank ingin menambahkan fitur bayar tagihan. Cukup buat satu kelas baru. MesinATM dan TransaksiBank tidak perlu diubah sama sekali.

```
class BayarTagihan(TransaksiBank):
    jenis = "Bayar PDAM"

    def validasi(self):
        return 0 < self.nominal <= self.rekening.saldo

    def eksekusi(self):
        self.rekening.saldo -= self.nominal


atm.proses([BayarTagihan(budi, 150_000)])

# Output:
# ATM Kampus Gunung Kelua
#   Bayar PDAM   Rp   150,000 -> BERHASIL
```

Inilah kekuatan polimorfisme: kode mudah diperluas tanpa mengubah yang sudah ada (maintainability).

Langkah 5: Notifikasi Multi-Kanal (Duck Typing)

Setelah transaksi, bank mengirim notifikasi lewat berbagai kanal. Ketiga kelas di bawah **tidak berhubungan lewat pewarisan**, tetapi semuanya punya method **kirim()**.

```
class Email:
    def __init__(self, alamat):
        self.alamat = alamat

    def kirim(self, pesan):
        print(f"  [EMAIL -> {self.alamat}] {pesan}")


class SMS:
    def __init__(self, nomor):
        self.nomor = nomor
```

```
    def kirim(self, pesan):
        print(f"  [SMS -> {self.nomor}] {pesan}")


class PushNotif:
    def __init__(self, device_id):
        self.device_id = device_id

    def kirim(self, pesan):
        print(f"  [PUSH -> {self.device_id}] {pesan}")


def beritahu_nasabah(kanal_list, pesan):
    """Duck typing: kanal apapun yang punya kirim() bisa dipakai."""
    for kanal in kanal_list:
        kanal.kirim(pesan)


kanal_budi = [Email("budi@mail.com"), SMS("0812-1234-5678"),
PushNotif("HP-BUDI-01")]
beritahu_nasabah(kanal_budi, "Transfer Rp300,000 ke Sari berhasil.")

# Output:
#   [EMAIL -> budi@mail.com] Transfer Rp300,000 ke Sari berhasil.
#   [SMS -> 0812-1234-5678] Transfer Rp300,000 ke Sari berhasil.
#   [PUSH -> HP-BUDI-01] Transfer Rp300,000 ke Sari berhasil.
```

Polimorfisme vs Pewarisan: Jangan Tertukar!

Keduanya terkait erat tetapi bukan hal yang sama:

Aspek	Pewarisan	Polimorfisme
Pertanyaan	"Kelas apa yang mewarisi siapa?"	"Antarmuka apa yang dipenuhi?"
Fokus	Struktur dan hubungan kelas	Perilaku dan cara dipanggil

Mekanisme	class Anak(Induk)	Method override, duck typing, ABC
Tujuan	Reuse kode dari induk	Satu antarmuka, banyak implementasi
Tanpa pewarisan?	Tidak mungkin	Bisa! (duck typing)

Polimorfisme bisa terjadi tanpa pewarisan (duck typing), tetapi pewarisan sangat membantu menegakkan kontrak antarmuka (ABC).

Kapan Gunakan ABC vs Duck Typing?

Situasi	ABC	Duck Typing
Ingin memaksa implementasi method tertentu	✅	❌
Bekerja dengan kelas dari library eksternal	❌	✅
Error terdeteksi saat instansiasi, bukan saat method dipanggil	✅	❌
Fleksibilitas maksimal tanpa hierarki kelas	❌	✅
Tim besar, butuh kontrak formal antar developer	✅	❌
Prototipe cepat atau skrip kecil	❌	✅

Panduan: Gunakan ABC ketika kamu mendefinisikan hierarki kelas sendiri (seperti TransaksiBank) dan ingin menjamin kontrak implementasi. Gunakan duck typing ketika bekerja dengan kelas dari pihak lain (seperti kanal notifikasi) atau ketika fleksibilitas lebih penting.

Anti-Pattern yang Harus Dihindari

```python
# [BURUK]: Memeriksa tipe secara manual, bukan polimorfisme!
def hitung_bunga_akun(akun):
    if type(akun) == Tabungan:
        return akun.saldo * 0.02
    elif type(akun) == Deposito:
        return akun.saldo * 0.06
```

```
    # Harus dimodifikasi setiap ada jenis akun baru!

# [BENAR]: Biarkan objek tahu cara menghitungnya sendiri
def hitung_bunga_akun(akun):
    return akun.hitung_bunga()   # Polimorfis, tidak perlu tahu tipe
```

Ringkasan Visual

```
Polimorfisme Python
│
├── Via Pewarisan (Inheritance Polymorphism)
│    ├── Kelas anak override method induk (Tabungan, Deposito)
│    ├── Satu nama method, banyak implementasi
│    └── Python memilih implementasi saat runtime (dynamic dispatch)
│
├── Duck Typing
│    ├── Tidak perlu hierarki kelas
│    ├── Cukup punya method dengan nama yang sama
│    └── "If it quacks like a duck, it's a duck"
│
├── Abstract Base Class (ABC)
│    ├── Paksa kelas anak mengimplementasikan method tertentu
│    ├── from abc import ABC, abstractmethod
│    └── Error saat membuat objek, bukan saat memanggil method
│
└── Operator Overloading
     ├── __add__, __sub__ untuk operasi aritmatika
     ├── __eq__, __lt__, __gt__ untuk perbandingan
     └── __len__, __str__ untuk fungsi built-in
```

Tabel Ringkasan:

Konsep	Sintaks/cara	Keterangan
Override method	Definisikan ulang di kelas anak	Nama method sama
Duck typing	Pastikan method ada	Tidak perlu pewarisan

ABC	class X(ABC) + @abstractmethod	Error saat instansiasi
Operator overloading	def __add__(self, other)	Mendefinisikan +
Cek tipe polimorfis	isinstance(obj, Kelas)	Mengenali hierarki